

```
!pip install fg-data-profiling
!pip install matplotlib
!pip install seaborn
!pip install pandas
!pip install numpy
```

```
Collecting fg-data-profiling
  Downloading fg_data_profiling-4.19.1-py2.py3-none-any.whl.metadata (23 kB)
Requirement already satisfied: scipy<1.17, >=1.8 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: pandas!=1.4.0, <3.0, >=1.5 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: matplotlib<=3.10, >=3.5 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: pydantic<3, >=2 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: PyYAML<6.1, >=6.0.3 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: Jinja2<3.2, >=3.1.6 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Collecting visions<0.8.2, >=0.7.5 (from visions[type_image_path]<0.8.2, >=0.7.5->fg-data-profiling)
  Downloading visions-0.8.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy<2.4, >=1.22 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Collecting minify-html==0.15.0 (from fg-data-profiling)
  Downloading minify_html-0.18.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (18 kB)
Collecting filetype==1.0.0 (from fg-data-profiling)
  Downloading filetype-1.2.0-py2.py3-none-any.whl.metadata (6.5 kB)
Collecting phik<0.13, >=0.12.5 (from fg-data-profiling)
  Downloading phik-0.12.5-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (5.6 kB)
Requirement already satisfied: requests<3, >=2.32.0 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: tqdm<5, >=4.66.3 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: seaborn<0.14, >=0.10.1 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Collecting multimethod<2, >=1.4 (from fg-data-profiling)
  Downloading multimethod-1.12-py3-none-any.whl.metadata (9.6 kB)
Requirement already satisfied: statsmodels<1, >=0.13.2 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: typeguard<5, >=4 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Collecting imagehash==4.3.2 (from fg-data-profiling)
  Downloading ImageHash-4.3.2-py2.py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: wordcloud>=1.9.4 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Collecting dacite<2, >=1.9 (from fg-data-profiling)
  Downloading dacite-1.9.2-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: numba<0.63, >=0.60 in /usr/local/lib/python3.12/dist-packages (from fg-data-profiling)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.12/dist-packages (from imagehash==4.3.2->fg-data-profiling)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from imagehash==4.3.2->fg-data-profiling)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2<3.2, >=3.1.6->fg-data-profiling)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib<=3.10, >=3.5->fg-data-profiling)
Requirement already satisfied: llvmlite<0.44, >=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba<0.63, >=0.60->fg-data-profiling)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas!=1.4.0, <3.0, >=1.5->fg-data-profiling)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas!=1.4.0, <3.0, >=1.5->fg-data-profiling)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.12/dist-packages (from phik<0.13, >=0.12.5->fg-data-profiling)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3, >=2->fg-data-profiling)
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3, >=2->fg-data-profiling)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic<3, >=2->fg-data-profiling)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3, >=2->fg-data-profiling)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages (from requests<3, >=2.32.0->fg-data-profiling)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3, >=2.32.0->fg-data-profiling)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3, >=2.32.0->fg-data-profiling)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3, >=2.32.0->fg-data-profiling)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmodels<1, >=0.13.2->fg-data-profiling)
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.12/dist-packages (from visions<0.8.2, >=0.7.5->fg-data-profiling)
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.12/dist-packages (from visions<0.8.2, >=0.7.5->fg-data-profiling)
Collecting puremagic (from visions<0.8.2, >=0.7.5->visions[type_image_path]<0.8.2, >=0.7.5->fg-data-profiling)
  Downloading puremagic-2.2.0-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: python-magic>=0.4.11 in /usr/local/lib/python3.12/dist-packages (from puremagic->visions[type_image_path]<0.8.2, >=0.7.5->fg-data-profiling)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import data_profiling
from data_profiling import ProfileReport
from google.colab import files
uploaded = files.upload()
```

[Choose files](#) Credit_Risk...er_Data.csv

Credit_Risk_Analysis_Uncleaned.xlsx - Raw_Borrower_Data.csv(text/csv) - 68156 bytes, last modified: 10/08/2026 - 100% done
Saving Credit_Risk_Analysis_Uncleaned.xlsx - Raw_Borrower_Data.csv to Credit_Risk_Analysis_Uncleaned.xlsx - Raw_Borrower_Data.csv

```
df=pd.read_csv('Credit_Risk_Analysis_Uncleaned.xlsx - Raw_Borrower_Data.csv')
#Profile = ProfileReport(df, title="Pandas Profiling Report", explorative=True)
#Profile.to_file("your_report.html")
```

```
df.shape
```

```
(600, 15)
```

```
df.dtypes
```

```

      0
borrower_id    object
full_name       object
age            object
annual_income   object
employment_status object
home_ownership  object
credit_score    float64
loan_amount     object
interest_rate   object
loan_purpose      object
origination_date object
days_past_due  float64
delinquency_status object
default_flag    object
notes          object

```

```
dtype: object
```

Identify and Handle Null Values

```
print("Missing values before handling:")
display(df.isnull().sum())
```

```
Missing values before handling:
```

```

      0
borrower_id    0
full_name      32
age            31
annual_income   34
employment_status 59
home_ownership  91
credit_score    26
loan_amount     42
interest_rate   31
loan_purpose      0
origination_date 25
days_past_due  22
delinquency_status 76
default_flag    28
notes          160

```

```
dtype: int64
```

Identify and Drop Duplicate Rows

```

initial_rows = df.shape[0]
duplicate_rows = df.duplicated().sum()
print(f"Number of duplicate rows found: {duplicate_rows}")

if duplicate_rows > 0:

```

```
df.drop_duplicates(inplace=True)
print(f"Duplicate rows dropped. New number of rows: {df.shape[0]}")
else:
    print("No duplicate rows found.")
```

```
Number of duplicate rows found: 0
No duplicate rows found.
```

✓ Clean and Convert `age` Column

```
# Convert 'age' to numeric, coercing errors to NaN
df['age'] = pd.to_numeric(df['age'], errors='coerce')

# Identify and count negative ages
negative_ages = df[df['age'] < 0]['age'].count()
print(f"Number of negative ages found: {negative_ages}")

# Handle negative ages: Convert them to their absolute value, assuming they are typos.
df['age'] = df['age'].abs()

# Impute missing ages with the median
df['age'].fillna(df['age'].median(), inplace=True)

print("Age column cleaned and missing values imputed with median.")
print(f"New data type of 'age' column: {df['age'].dtype}")
print("Missing values in 'age' after cleaning:")
display(df['age'].isnull().sum())
```

```
Number of negative ages found: 22
Age column cleaned and missing values imputed with median.
New data type of 'age' column: float64
Missing values in 'age' after cleaning:
/tmp/ipykernel_746/848910342.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on wh
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df['age'].fillna(df['age'].median(), inplace=True)
np.int64(0)
```

✓ Clean and Convert Numerical Columns

```
numerical_cols_to_clean = ['annual_income', 'loan_amount', 'interest_rate']

for col in numerical_cols_to_clean:
    # Remove non-numeric characters like '$', ',', '%' and convert to numeric
    df[col] = df[col].astype(str).str.replace('$', '', regex=False).str.replace(',', '', regex=False).str.replace(
    df[col] = pd.to_numeric(df[col], errors='coerce')

    # Impute missing values with the median
    median_value = df[col].median()
    df[col].fillna(median_value, inplace=True)
    print(f"Column '{col}' cleaned, converted to numeric, and missing values imputed with median ({median_value}:
    print(f"New data type of '{col}' column: {df[col].dtype}")
    print(f"Missing values in '{col}' after cleaning: {df[col].isnull().sum()}\n")

print("Data types after cleaning numerical columns:")
display(df[numerical_cols_to_clean].dtypes)
```

```
Column 'annual_income' cleaned, converted to numeric, and missing values imputed with median (88348.21).
New data type of 'annual_income' column: float64
Missing values in 'annual_income' after cleaning: 0
```

```
Column 'loan_amount' cleaned, converted to numeric, and missing values imputed with median (24000.00).
New data type of 'loan_amount' column: float64
Missing values in 'loan_amount' after cleaning: 0
```

```
Column 'interest_rate' cleaned, converted to numeric, and missing values imputed with median (0.27).
New data type of 'interest_rate' column: float64
Missing values in 'interest_rate' after cleaning: 0
```

```
Data types after cleaning numerical columns:
```

```
/tmp/ipykernel_746/3886049117.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
```

```
df[col].fillna(median_value, inplace=True)
/tmp/ipykernel_746/3886049117.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
```

```
df[col].fillna(median_value, inplace=True)
/tmp/ipykernel_746/3886049117.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
```

```
df[col].fillna(median_value, inplace=True)
```

```
0
```

```
annual_income    float64
```

```
loan_amount      float64
```

```
interest_rate    float64
```

```
dtype: object
```

▼ Convert `origination_date` to Datetime

```
# Convert 'origination_date' to datetime, coercing errors to NaT
df['origination_date'] = pd.to_datetime(df['origination_date'], errors='coerce')

print(f"New data type of 'origination_date' column: {df['origination_date'].dtype}")
print("Missing values in 'origination_date' after conversion:")
display(df['origination_date'].isnull().sum())
```

```
# Impute missing 'origination_date' values with the mode (most frequent date)
# It's important to do this after conversion to datetime for correct imputation
if df['origination_date'].isnull().sum() > 0:
    mode_date = df['origination_date'].mode()[0]
    df['origination_date'].fillna(mode_date, inplace=True)
    print(f"Missing 'origination_date' values imputed with mode: {mode_date}")
    print("Missing values in 'origination_date' after imputation:")
    display(df['origination_date'].isnull().sum())
```

```
New data type of 'origination_date' column: datetime64[ns]
```

```
Missing values in 'origination_date' after conversion:
```

```
np.int64(487)
```

```
Missing 'origination_date' values imputed with mode: 2021-06-17 00:00:00
```

```
Missing values in 'origination_date' after imputation:
```

```
/tmp/ipykernel_746/4021532928.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
```

```
df['origination_date'].fillna(mode_date, inplace=True)
np.int64(0)
```

▼ Convert `default_flag` to Boolean or Categorical

```
# Convert 'default_flag' to boolean, mapping 'TRUE' to True and 'FALSE' to False
# Coerce errors will turn other values into NaN
df['default_flag'] = df['default_flag'].map({'TRUE': True, 'FALSE': False}).astype(bool)

# Impute missing 'default_flag' values with the mode (most frequent value)
if df['default_flag'].isnull().sum() > 0:
    mode_default_flag = df['default_flag'].mode()[0]
    df['default_flag'].fillna(mode_default_flag, inplace=True)
    print(f"Missing 'default_flag' values imputed with mode: {mode_default_flag}")

print(f"New data type of 'default_flag' column: {df['default_flag'].dtype}")
print("Missing values in 'default_flag' after cleaning:")
display(df['default_flag'].isnull().sum())
```

```
New data type of 'default_flag' column: bool
Missing values in 'default_flag' after cleaning:
np.int64(0)
```

▼ Address **notes** Column

```
if 'notes' in df.columns:
    df.drop('notes', axis=1, inplace=True)
    print("The 'notes' column has been dropped due to many missing values and unstructured text.")
else:
    print("The 'notes' column does not exist.")

print("Current DataFrame shape after dropping 'notes' column:", df.shape)
```

```
The 'notes' column has been dropped due to many missing values and unstructured text.
Current DataFrame shape after dropping 'notes' column: (600, 14)
```

▼ Review and Clean Categorical Columns

▼ Clean **employment_status** Column

```
print("Unique values in 'employment_status' before cleaning:")
display(df['employment_status'].value_counts(dropna=False))

# Standardize common variations (e.g., 'full_time' and 'Full-Time')
df['employment_status'] = df['employment_status'].replace({
    'full_time': 'Full-Time',
    'part_time': 'Part-Time'
})

# Impute missing values with the mode
if df['employment_status'].isnull().sum() > 0:
    mode_employment_status = df['employment_status'].mode()[0]
    df['employment_status'].fillna(mode_employment_status, inplace=True)
    print(f"Missing 'employment_status' values imputed with mode: {mode_employment_status}")

print("\nUnique values in 'employment_status' after cleaning:")
display(df['employment_status'].value_counts(dropna=False))
```

Unique values in 'employment_status' before cleaning:

	count
employment_status	
NaN	59
Contractor	52
Self Emp	50
full_time	50
RETIRED	47
Retired	45
unemployed	45
Full Time	40
FT	37
Self-Employed	37
Full-Time	36
PT	35
UNEMPLOYED	34
Part-time	33

dtype: int64

Missing 'employment_status' values imputed with mode: Full-Time

Unique values in 'employment_status' after cleaning:

/tmp/ipykernel_746/1205805722.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is performed is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df['employment_status'].fillna(mode_employment_status, inplace=True)
```

	count
employment_status	
Full-Time	145
Contractor	52
Self Emp	50
RETIRED	47
Retired	45
unemployed	45
Full Time	40
Self-Employed	37
FT	37
PT	35
UNEMPLOYED	34
Part-time	33

dtype: int64

Save and View Cleaned Dataset

```
# Save the cleaned DataFrame to a new CSV file
df.to_csv('cleaned_credit_risk_data.csv', index=False)
print("Cleaned dataset saved to 'cleaned_credit_risk_data.csv'")

# Display the first 5 rows of the cleaned DataFrame
print("\nFirst 5 rows of the cleaned dataset:")
display(df.head())
```

Cleaned dataset saved to 'cleaned_credit_risk_data.csv'

First 5 rows of the cleaned dataset:

	borrower_id	full_name	age	annual_income	employment_status	home_ownership	credit_score	loan_amount	interest_rate
0	BW-1001	Diana Martinez	50.0	101449.23	Contractor	Mortgage	180.0	8000.0	12.5
1	BW-1002	Jack Moore	46.0	134363.11	Contractor	NaN	544.0	12000.0	13.2
2	BW-1003	Robert Davis	5.0	143424.01	Full-Time	NaN	775.0	11000.0	11.8
3	BW-1004	Xavier Thomas	45.0	58502.22	Contractor	OTHER	703.0	14000.0	14.1
4	BW-1005	Ivy White	24.0	54370.00	Full-Time	Mortgage	643.0	46000.0	12.9

```
# Display the complete cleaned DataFrame
print("Complete cleaned dataset:")
display(df)
```

Complete cleaned dataset:

	borrower_id	full_name	age	annual_income	employment_status	home_ownership	credit_score	loan_amount	interest_rate
0	BW-1001	Diana Martinez	50.0	101449.23	Contractor	Mortgage	180.0	8000.0	12.5
1	BW-1002	Jack Moore	46.0	134363.11	Contractor	NaN	544.0	12000.0	13.2
2	BW-1003	Robert Davis	5.0	143424.01	Full-Time	NaN	775.0	11000.0	11.8
3	BW-1004	Xavier Thomas	45.0	58502.22	Contractor	OTHER	703.0	14000.0	14.1
4	BW-1005	Ivy White	24.0	54370.00	Full-Time	Mortgage	643.0	46000.0	12.9
...	...	...	...	...	...	...	...	...	...
595	BW-1596	LIAM MARTIN	58.0	108591.38	Contractor	NaN	734.0	35000.0	13.5
596	BW-1597	Bob Jackson	33.0	147854.80	Part-time	Mortgage	681.0	24000.0	12.7
597	BW1598	Frank Smith	37.0	51707.05	Full-Time	Rent	698.0	44000.0	13.0
598	BW-1599	Ivy Wilson	57.0	101496.97	Full Time	MORTGAGE	833.0	7000.0	11.5
599	bw_1600	Robert Thompson	48.0	64722.83	Part-time	NaN	837.0	40000.0	13.8

600 rows x 14 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

Double-click (or enter) to edit

```
#from google.colab import files

# Download the cleaned CSV file
#files.download('cleaned_credit_risk_data.csv')
#import matplotlib.pyplot as plt
#import seaborn as sns
```

Review DataFrame Information and Statistics

```
print('DataFrame Info:')
df.info()
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   borrower_id           600 non-null    object
1   full_name             568 non-null    object
2   age                   600 non-null    float64
3   annual_income         600 non-null    float64
```

```

4  employment_status  600 non-null  object
5  home_ownership     509 non-null  object
6  credit_score       574 non-null  float64
7  loan_amount        600 non-null  float64
8  interest_rate      600 non-null  float64
9  loan_purpose         600 non-null  object
10 origination_date   600 non-null  datetime64[ns]
11 days_past_due      578 non-null  float64
12 delinquency_status 524 non-null  object
13 default_flag       600 non-null  bool
dtypes: bool(1), datetime64[ns](1), float64(6), object(6)
memory usage: 61.7+ KB

```

```

print('\nDataFrame Description (including all column types):')
display(df.describe(include='all'))

```

DataFrame Description (including all column types):

	borrower_id	full_name	age	annual_income	employment_status	home_ownership	credit_score	loan_amount
count	600	568	600.000000	6.000000e+02	600	509	574.000000	600.000000
unique	598	447	NaN	NaN	12	9	NaN	NaN
top	BW-100252	Michael Johnson	NaN	NaN	Full-Time	rent	NaN	NaN
freq	2	4	NaN	NaN	145	67	NaN	NaN
mean	NaN	NaN	46.941667	2.175121e+07	NaN	NaN	677.290941	24415.8333
min	NaN	NaN	5.000000	-1.500000e+04	NaN	NaN	180.000000	-7500.0000
25%	NaN	NaN	33.000000	6.177334e+04	NaN	NaN	611.250000	13000.0000
50%	NaN	NaN	45.000000	8.834821e+04	NaN	NaN	688.000000	24000.0000
75%	NaN	NaN	57.000000	1.165706e+05	NaN	NaN	776.000000	36000.0000
max	NaN	NaN	145.000000	1.000000e+09	NaN	NaN	950.000000	50000.0000
std	NaN	NaN	24.531638	1.457016e+08	NaN	NaN	151.987625	14462.3166

✓ Applying further cleaning rules based on domain knowledge

```

# Original number of rows for comparison
initial_rows_age = df.shape[0]

# Handle age column
# Remove entire rows where age >= 85
df = df[df['age'] < 85]
print(f"Removed {initial_rows_age - df.shape[0]} rows where age was 85 or above.")

# For the remaining rows, set 'Ineligible' if age <= 16
df.loc[df['age'] <= 16, 'age'] = 'Ineligible'
print("Set 'age' to 'Ineligible' for individuals aged 16 or below.")

# Handle annual_income column: replace values <= 0 with 0
df.loc[df['annual_income'] <= 0, 'annual_income'] = 0
print("Replaced 'annual_income' values less than or equal to 0 with 0.")

# Handle loan_amount column: replace values <= 0 with 0
df.loc[df['loan_amount'] <= 0, 'loan_amount'] = 0
print("Replaced 'loan_amount' values less than or equal to 0 with 0.")

# Handle days_past_due column: replace values <= 0 with 0
df.loc[df['days_past_due'] <= 0, 'days_past_due'] = 0
print("Replaced 'days_past_due' values less than or equal to 0 with 0.")

print("\nCleaning complete. Displaying the first 5 rows and info of the updated DataFrame:")
display(df.head())
df.info()

```


Removed 23 rows where age was 85 or above.
 Set 'age' to 'Ineligible' for individuals aged 16 or below.
 Replaced 'annual_income' values less than or equal to 0 with 0.
 Replaced 'loan_amount' values less than or equal to 0 with 0.
 Replaced 'days_past_due' values less than or equal to 0 with 0.

Cleaning complete. Displaying the first 5 rows and info of the updated DataFrame:

/tmp/ipykernel_746/3858218171.py:10: FutureWarning: Setting an item of incompatible dtype is deprecated and will
 df.loc[df['age'] <= 16, 'age'] = 'Ineligible'

	borrower_id	full_name	age	annual_income	employment_status	home_ownership	credit_score	loan_amount	int
0	BW-1001	Diana Martinez	50.0	101449.23	Contractor	Mortgage	180.0	8000.0	
1	BW-1002	Jack Moore	46.0	134363.11	Contractor	NaN	544.0	12000.0	
2	BW-1003	Robert Davis	Ineligible	143424.01	Full-Time	NaN	775.0	11000.0	
3	BW-1004	Xavier Thomas	45.0	58502.22	Contractor	OTHER	703.0	14000.0	
4	BW-1005	Ivy White	24.0	54370.00	Full-Time	Mortgage	643.0	46000.0	

<class 'pandas.core.frame.DataFrame'>

Index: 577 entries, 0 to 599

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	borrower_id	577 non-null	object
1	full_name	545 non-null	object
2	age	577 non-null	object
3	annual_income	577 non-null	float64
4	employment_status	577 non-null	object
5	home_ownership	489 non-null	object
6	credit_score	552 non-null	float64
7	loan_amount	577 non-null	float64
8	interest_rate	577 non-null	float64
9	loan_purpose	577 non-null	object
10	origination_date	577 non-null	datetime64[ns]
11	days_past_due	558 non-null	float64
12	delinquency_status	503 non-null	object
13	default_flag	577 non-null	bool

dtypes: bool(1), datetime64[ns](1), float64(5), object(7)

memory usage: 63.7+ KB

Visual Data Exploration: Max, Min, and Mode

Numerical Columns Analysis

```
numerical_cols = ['annual_income', 'credit_score', 'loan_amount', 'interest_rate', 'days_past_due']
```

```
print("--- Summary Statistics for Numerical Columns ---")
```

```
for col in numerical_cols:
```

```
    print(f"\nColumn: {col}")
```

```
    print(f"  Min: {df[col].min():.2f}")
```

```
    print(f"  Max: {df[col].max():.2f}")
```

```
    print(f"  Mode: {df[col].mode()[0]:.2f}") # Displaying the first mode if multiple exist
```

```
    print(f"  Median: {df[col].median():.2f}")
```

```
# Plotting distributions for numerical columns
```

```
fig, axes = plt.subplots(len(numerical_cols), 1, figsize=(10, 5 * len(numerical_cols)))
```

```
fig.suptitle('Distribution of Numerical Columns', y=1.02, fontsize=16)
```

```
for i, col in enumerate(numerical_cols):
```

```
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i])
```

```
    axes[i].set_title(f'Distribution of {col}')
```

```
    axes[i].set_xlabel(col)
```

```
    axes[i].set_ylabel('Frequency')
```

```
plt.tight_layout()
```

```
plt.show()
```


--- Summary Statistics for Numerical Columns ---

Column: annual_income

Min: 0.00

Max: 999999999.00

Mode: 88348.21

Median: 88348.21

Column: credit_score

Min: 180.00

Max: 950.00

Mode: 180.00

Median: 688.00

Column: loan_amount

Min: 0.00

Max: 50000.00

Mode: 24000.00

Median: 24000.00

Column: interest_rate

Min: 0.05

Max: 28.98

Mode: 0.27

Median: 0.27

Column: days_past_due

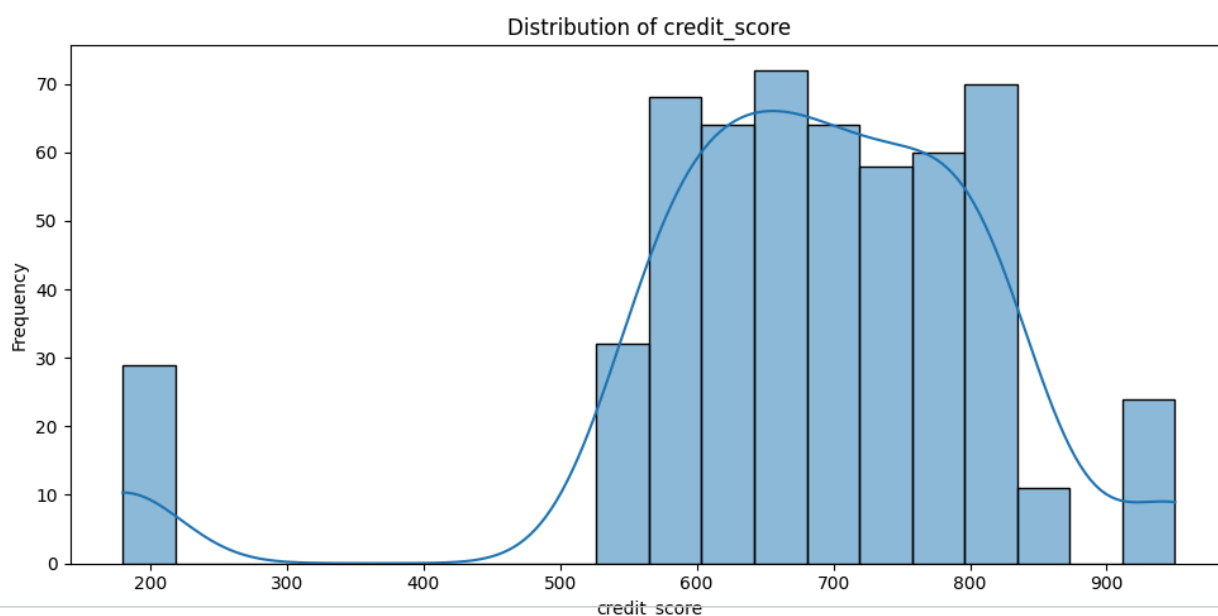
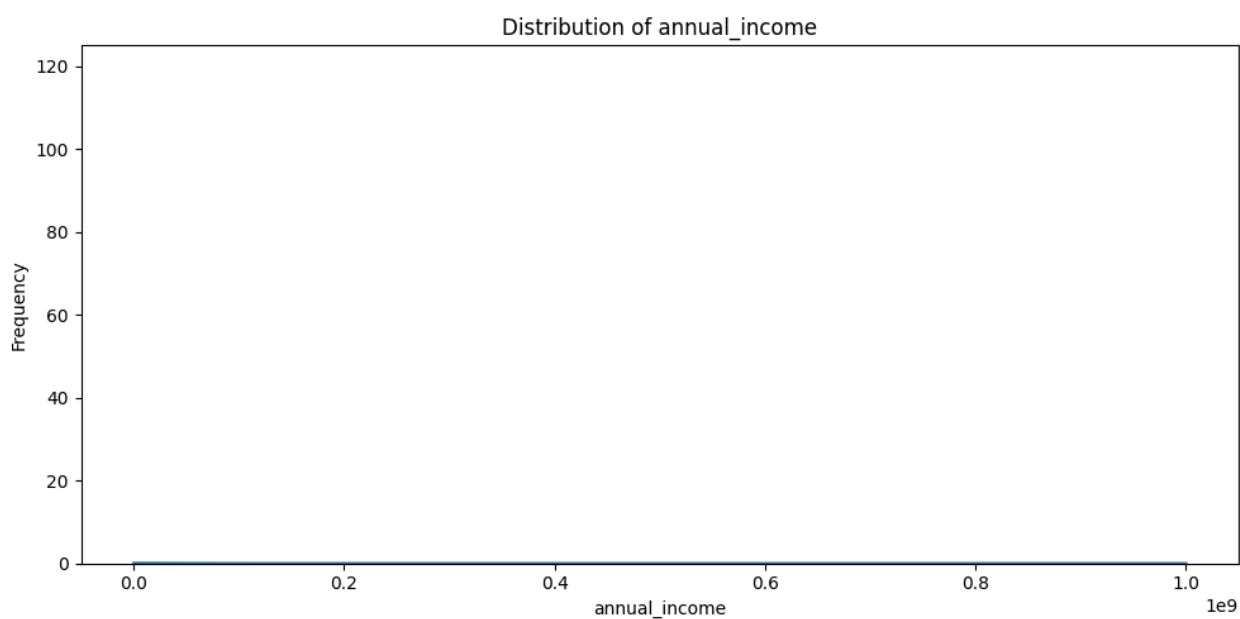
Min: 0.00

Max: 180.00

Mode: 0.00

Median: 0.00

Distribution of Numerical Columns



'age' Column Analysis

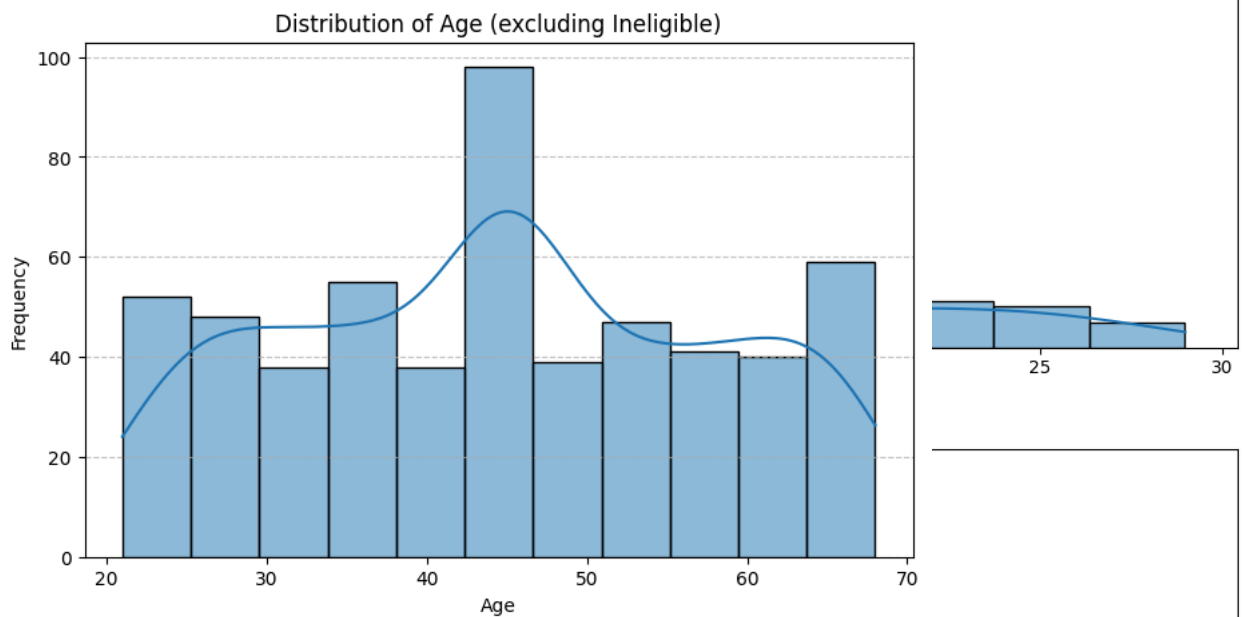
Distribution of loan_amount

```
# Separate numeric ages from 'Ineligible'
numeric_ages = pd.to_numeric(df['age'], errors='coerce').dropna()
ineligible_count = df[df['age'] == 'Ineligible'].shape[0]

print("---- Summary Statistics for 'age' Column ----")
if not numeric_ages.empty:
    print(f"  Min (numeric): {numeric_ages.min():.0f}")
    print(f"  Max (numeric): {numeric_ages.max():.0f}")
    print(f"  Mode (numeric): {numeric_ages.mode()[0]:.0f}")
else:
    print("No numeric ages found for min/max/mode calculation.")
print(f"  'Ineligible' count: {ineligible_count}")

# Plotting distribution for numeric ages
if not numeric_ages.empty:
    plt.figure(figsize=(8, 5))
    sns.histplot(numeric_ages, kde=True)
    plt.title('Distribution of Age (excluding Ineligible)')
    plt.xlabel('Age')
    plt.ylabel('Frequency')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()
else:
    print("No numeric ages to plot.")
```

```
---- Summary Statistics for 'age' Column ----
Min (numeric): 21
Max (numeric): 68
Mode (numeric): 45
'Ineligible' count: 22
```



Categorical Columns Analysis

```
categorical_cols = ['employment_status', 'home_ownership', 'loan_purpose', 'delinquency_status', 'default_flag']

print("---- Mode for Categorical Columns ----")
for col in categorical_cols:
    print(f"\nColumn: {col}")
    if not df[col].dropna().empty:
        print(f"  Mode: {df[col].mode()[0]}")
    else:
        print("  No non-null values to determine mode.")

# Plotting value counts for categorical columns
fig, axes = plt.subplots(len(categorical_cols), 1, figsize=(10, 5 * len(categorical_cols)))
fig.suptitle('Value Counts for Categorical Columns', y=1.02, fontsize=16)

for i, col in enumerate(categorical_cols):
    sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
    axes[i].set_title(f'Count of {col}')
    axes[i].set_xlabel('Count')
```

```
plt.tight_layout()  
plt.show()
```



```
--- Mode for Categorical Columns ---
```

```
Column: employment_status
Mode: Full-Time
```

```
Column: home_ownership
Mode: rent
```

```
Column: loan_purpose
Mode: Auto Loan
```

```
Column: delinquency_status
Mode: Defaulted
```

```
Column: default_flag
Mode: True
```

```
/tmp/ipykernel_746/249559022.py:17: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
/tmp/ipykernel_746/249559022.py:17: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
/tmp/ipykernel_746/249559022.py:17: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
/tmp/ipykernel_746/249559022.py:17: FutureWarning:
```

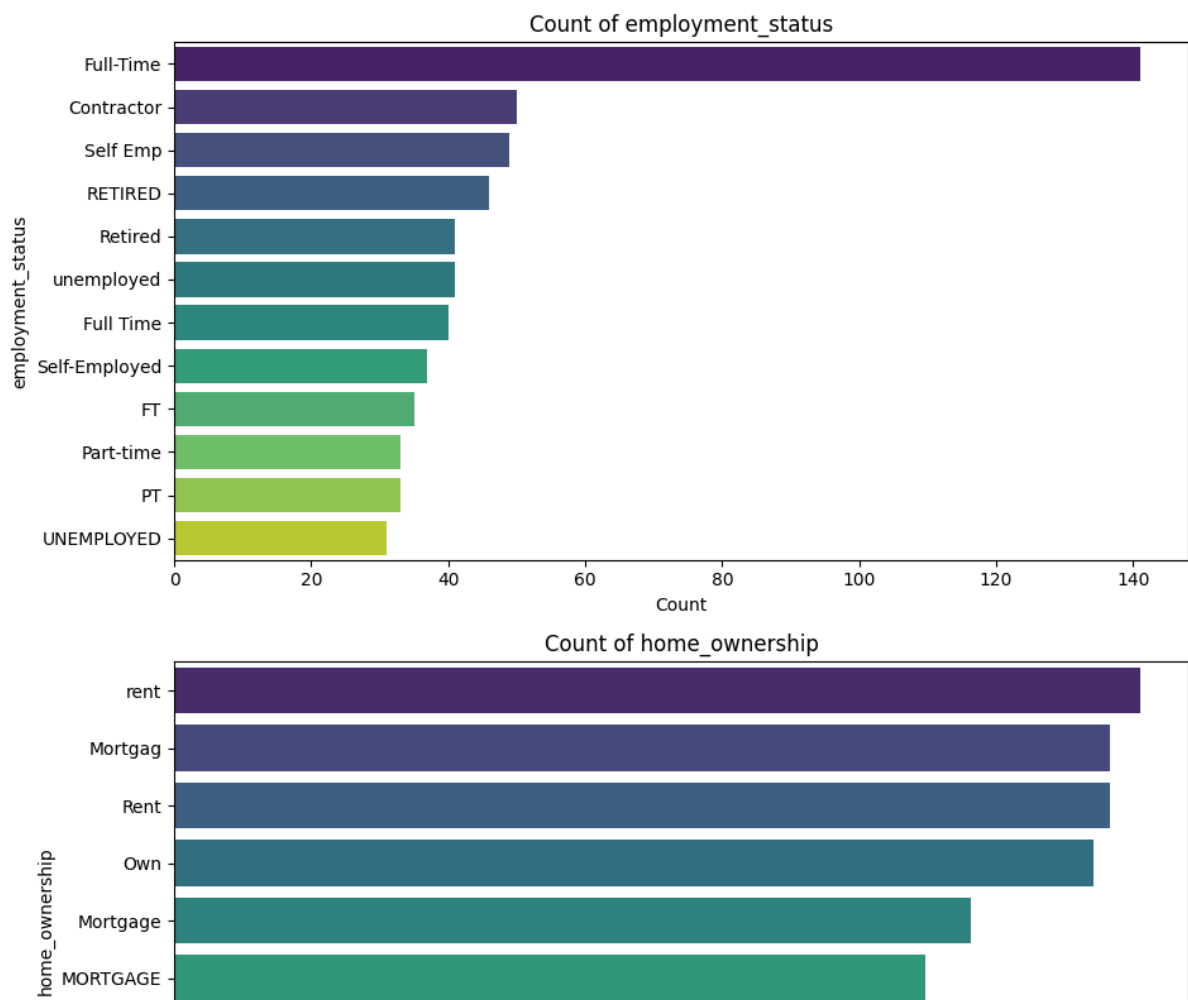
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

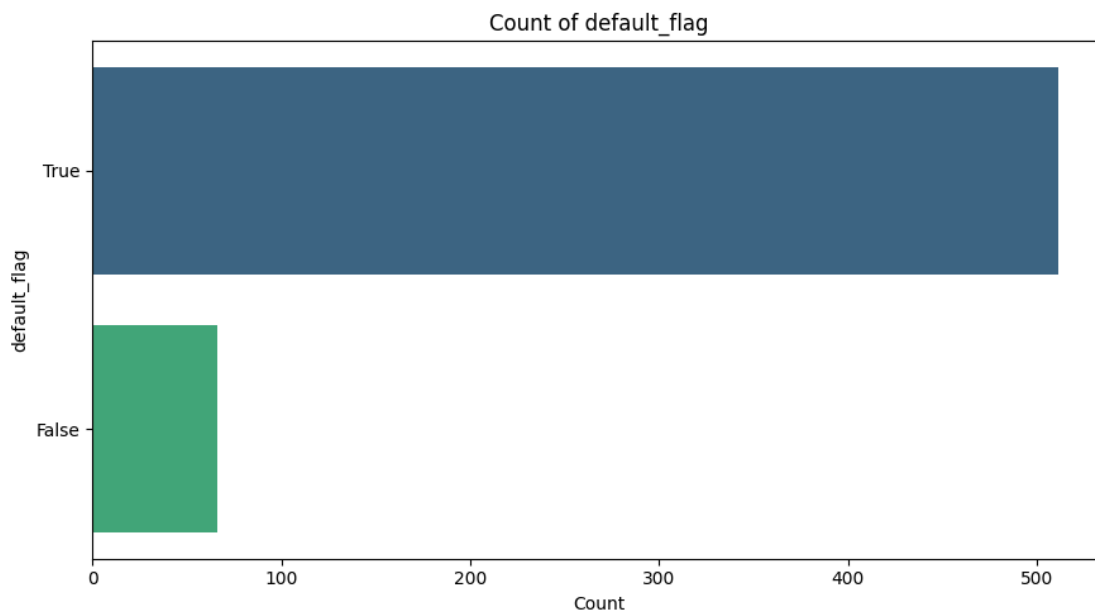
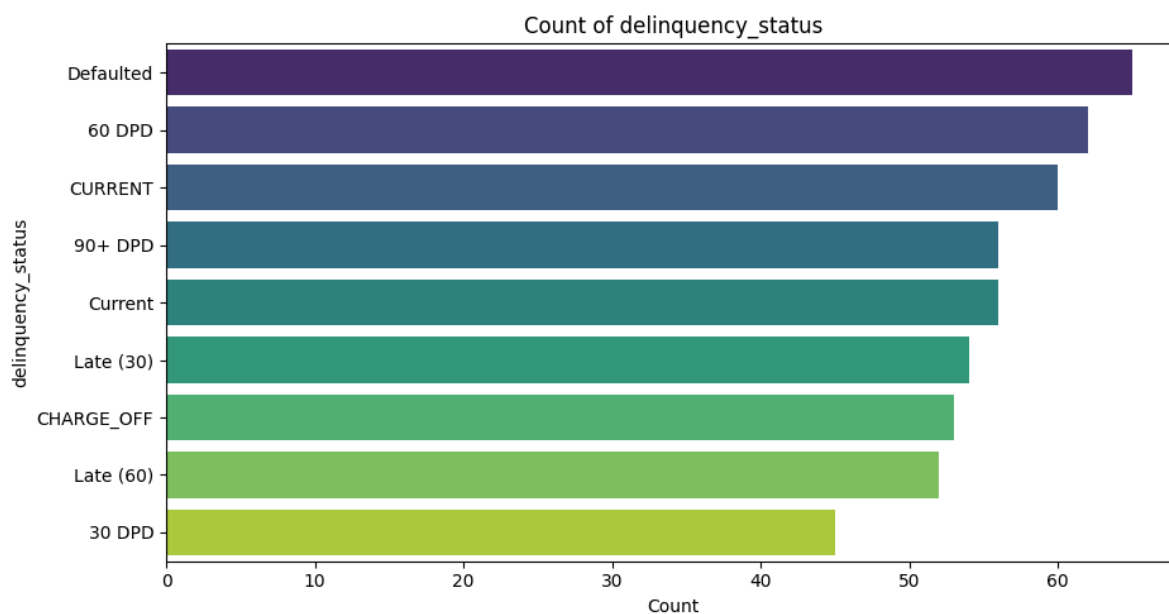
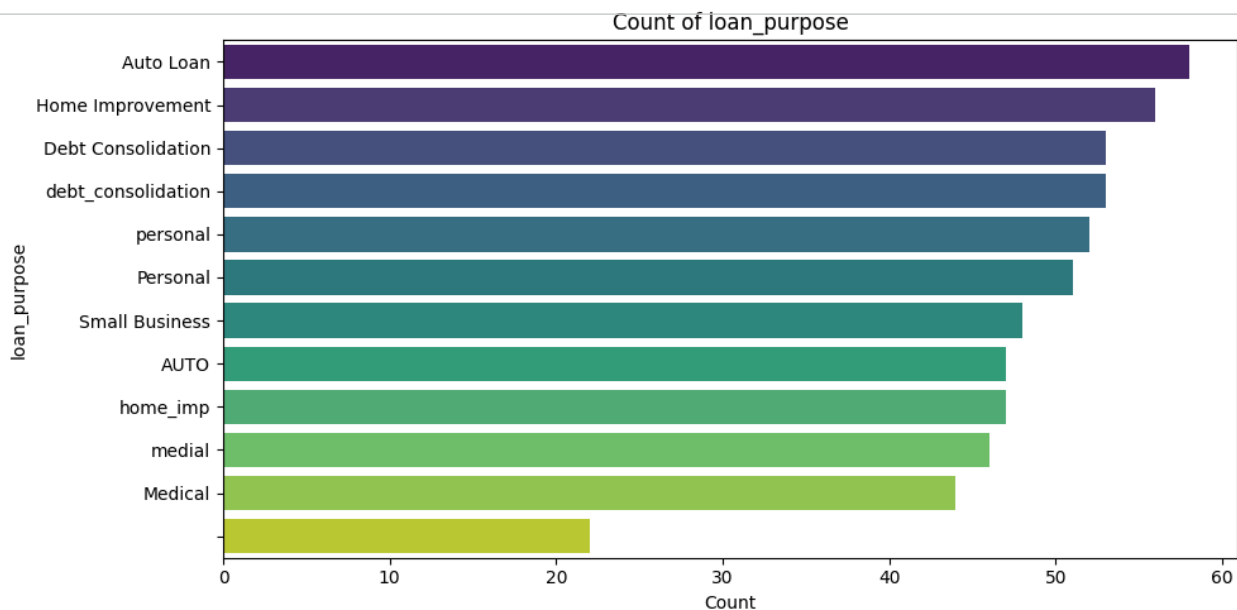
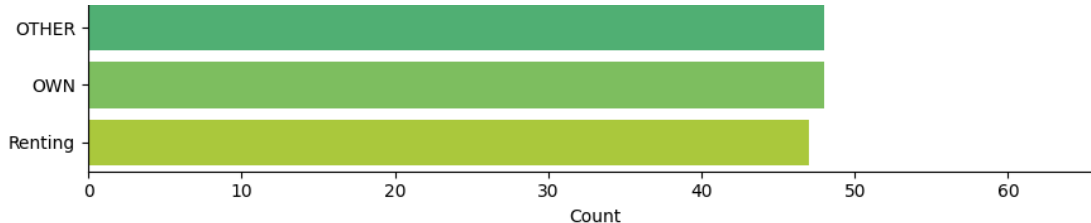
```
sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
/tmp/ipykernel_746/249559022.py:17: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
sns.countplot(y=df[col].dropna(), order=df[col].value_counts().index, ax=axes[i], palette='viridis')
```

Value Counts for Categorical Columns



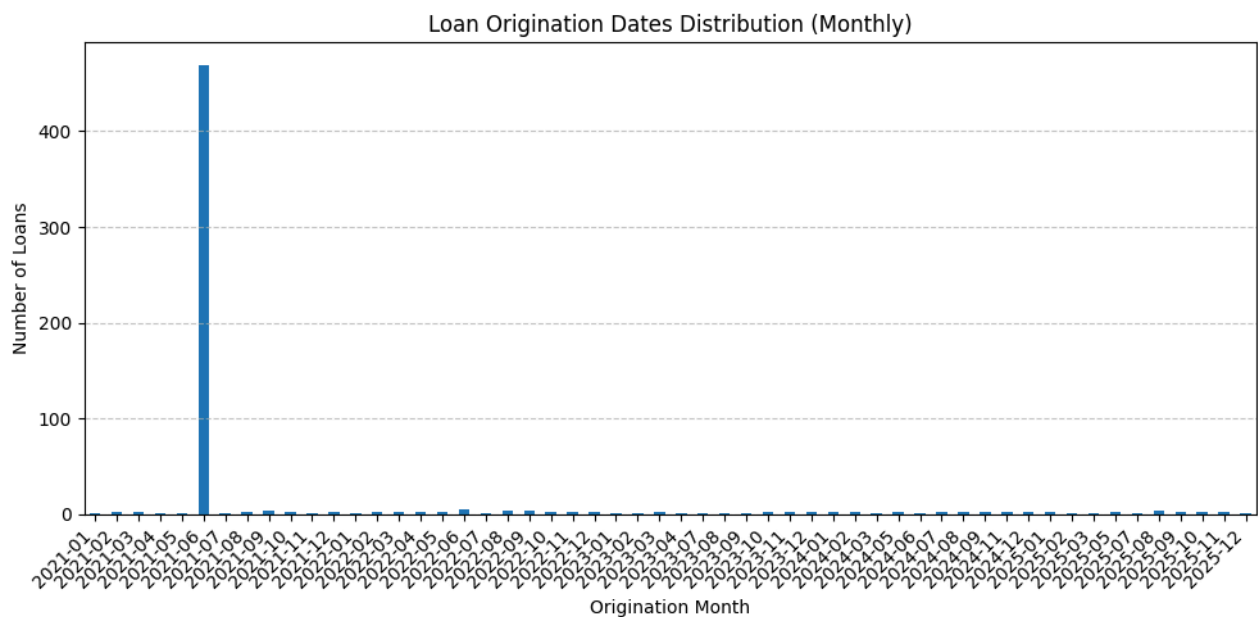


▼ 'origination_date' Column Analysis

```
print("--- Summary Statistics for 'origination_date' Column ---")
print(f"  Min Date: {df['origination_date'].min()}")
print(f"  Max Date: {df['origination_date'].max()}")
print(f"  Mode Date: {df['origination_date'].mode()[0]}")

# Plotting a time series distribution of origination_date
plt.figure(figsize=(10, 5))
df['origination_date'].dt.to_period('M').value_counts().sort_index().plot(kind='bar')
plt.title('Loan Origination Dates Distribution (Monthly)')
plt.xlabel('Origination Month')
plt.ylabel('Number of Loans')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
--- Summary Statistics for 'origination_date' Column ---
Min Date: 2021-01-17 00:00:00
Max Date: 2025-12-13 00:00:00
Mode Date: 2021-06-17 00:00:00
```



▼ Correlation Analysis of Numerical Columns

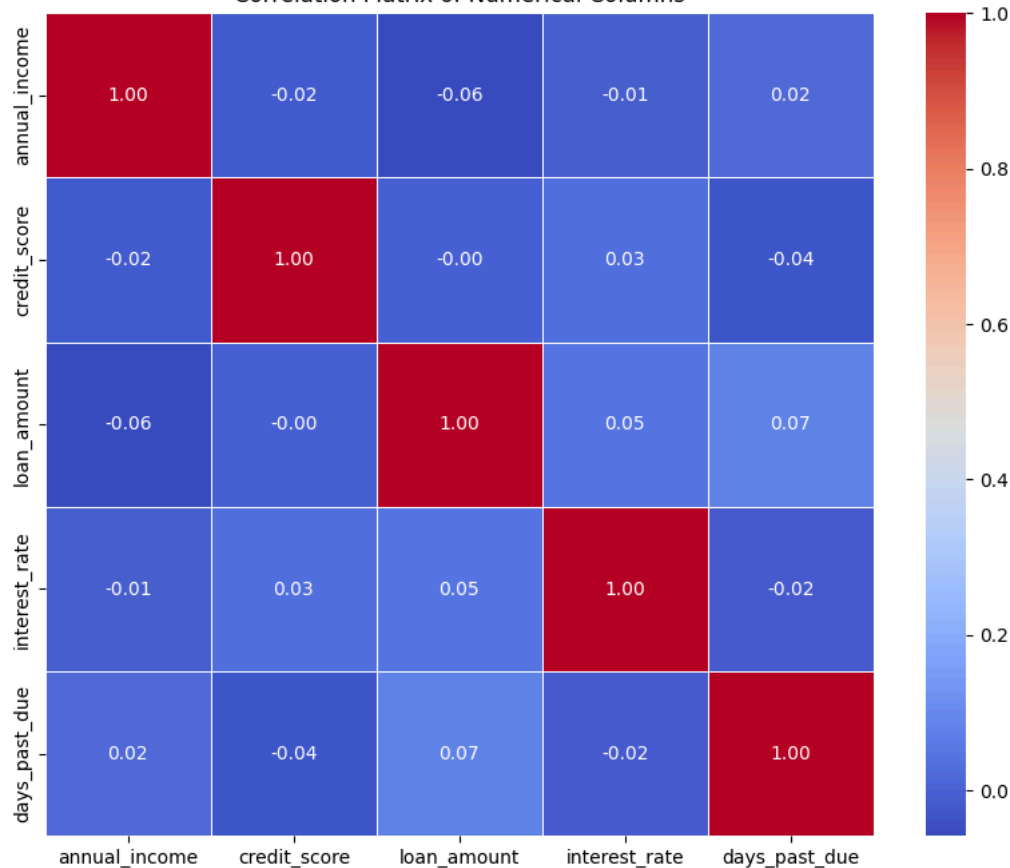
```
print("--- Correlation Matrix for Numerical Columns ---")
correlation_matrix = df[numerical_cols].corr()
display(correlation_matrix)

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of Numerical Columns')
plt.show()
```

--- Correlation Matrix for Numerical Columns ---

	annual_income	credit_score	loan_amount	interest_rate	days_past_due
annual_income	1.000000	-0.018162	-0.058650	-0.009945	0.015849
credit_score	-0.018162	1.000000	-0.001987	0.025948	-0.036419
loan_amount	-0.058650	-0.001987	1.000000	0.047135	0.074618
interest_rate	-0.009945	0.025948	0.047135	1.000000	-0.017056
days_past_due	0.015849	-0.036419	0.074618	-0.017056	1.000000

Correlation Matrix of Numerical Columns



Next steps: [Generate code with correlation_matrix](#) [New interactive sheet](#)

▼ Data Preparation for Logistic Regression

Before running the logistic regression, we need to ensure all features are numerical and handle any remaining missing values. We will also encode categorical features.

```
print("Missing values before final imputation:")
display(df.isnull().sum())

# Impute remaining numerical columns with median
# 'credit_score' and 'days_past_due' still have missing values
for col in ['credit_score', 'days_past_due']:
    if df[col].isnull().any():
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Imputed '{col}' with median: {median_val}")

# Impute remaining categorical columns with mode
# 'home_ownership', 'delinquency_status', 'full_name' still have missing values
for col in ['home_ownership', 'delinquency_status']:
    if df[col].isnull().any():
        mode_val = df[col].mode()[0]
        df[col].fillna(mode_val, inplace=True)
        print(f"Imputed '{col}' with mode: {mode_val}")

# Handle 'age' column: convert 'Ineligible' to NaN and then impute with median of numeric ages
# First, coerce 'age' to numeric, turning 'Ineligible' into NaN
df['age'] = pd.to_numeric(df['age'], errors='coerce')
```

```
# Now, impute any NaN values (including those from 'Ineligible') with the median of the numeric ages
if df['age'].isnull().any():
    median_age = df['age'].median()
    df['age'].fillna(median_age, inplace=True)
    print(f"Imputed 'age' (including 'Ineligible' converted to NaN) with median: {median_age}")

# Drop 'full_name' and 'borrower_id' as they are identifiers and not suitable for modeling
df.drop(columns=['full_name', 'borrower_id'], inplace=True)
print("Dropped 'full_name' and 'borrower_id' columns.")

# 'origination_date' is a datetime object, extracting year/month might be useful, or drop for simplicity in this
# For simplicity, we'll drop it for now in this initial model.
df.drop(columns=['origination_date'], inplace=True)
print("Dropped 'origination_date' column for initial model.")

print("\nMissing values after final imputation:")
display(df.isnull().sum())
```

Missing values before final imputation:

	0
borrower_id	0
full_name	32
age	0
annual_income	0
employment_status	0
home_ownership	88
credit_score	25
loan_amount	0
interest_rate	0
loan_purpose	0
origination_date	0
days_past_due	19
delinquency_status	74
default_flag	0

dtype: int64

Imputed 'credit_score' with median: 688.0

Imputed 'days_past_due' with median: 0.0

Imputed 'home_ownership' with mode: rent

Imputed 'delinquency_status' with mode: Defaulted

Imputed 'age' (including 'Ineligible' converted to NaN) with median: 45.0

Dropped 'full_name' and 'borrower_id' columns.

Dropped 'origination_date' column for initial model.

Missing values after final imputation:

/tmp/ipykernel_746/291553487.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is performed is a copy.
 df[col].method(value, inplace=True)

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)

df[col].fillna(median_val, inplace=True)

/tmp/ipykernel_746/291553487.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is performed is a copy.
 df[col].method(value, inplace=True)

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)

df[col].fillna(mode_val, inplace=True)

/tmp/ipykernel_746/291553487.py:28: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is performed is a copy.
 df[col].method(value, inplace=True)

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)

df['age'].fillna(median_age, inplace=True)

	0
age	0
annual_income	0
employment_status	0
home_ownership	0
credit_score	0
loan_amount	0
interest_rate	0
loan_purpose	0
days_past_due	0
delinquency_status	0
default_flag	0

dtype: int64

One-Hot Encode Categorical Features

```
categorical_features = ['employment_status', 'home_ownership', 'loan_purpose', 'delinquency_status']
df = pd.get_dummies(df, columns=categorical_features, drop_first=True)
```

```
print("DataFrame head after one-hot encoding:")
display(df.head())
print("DataFrame info after one-hot encoding:")
df.info()
```

DataFrame head after one-hot encoding:

	age	annual_income	credit_score	loan_amount	interest_rate	days_past_due	default_flag	employment_status_F
0	50.0	101449.23	180.0	8000.0	17.9000	0.0	True	Fals
1	46.0	134363.11	544.0	12000.0	0.1536	0.0	True	Fals
2	45.0	143424.01	775.0	11000.0	0.2719	45.0	True	Fals
3	45.0	58502.22	703.0	14000.0	0.2694	60.0	True	Fals
4	24.0	54370.00	643.0	46000.0	0.1732	0.0	True	Fals

5 rows x 45 columns

DataFrame info after one-hot encoding:

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 577 entries, 0 to 599

Data columns (total 45 columns):

#	Column	Non-Null Count	Dtype
0	age	577 non-null	float64
1	annual_income	577 non-null	float64
2	credit_score	577 non-null	float64
3	loan_amount	577 non-null	float64
4	interest_rate	577 non-null	float64
5	days_past_due	577 non-null	float64
6	default_flag	577 non-null	bool
7	employment_status_FT	577 non-null	bool
8	employment_status_Full Time	577 non-null	bool
9	employment_status_Full-Time	577 non-null	bool
10	employment_status_PT	577 non-null	bool
11	employment_status_Part-time	577 non-null	bool
12	employment_status_RETIRED	577 non-null	bool
13	employment_status_Retired	577 non-null	bool
14	employment_status_Self Emp	577 non-null	bool
15	employment_status_Self-Employed	577 non-null	bool
16	employment_status_UNEMPLOYED	577 non-null	bool
17	employment_status_unemployed	577 non-null	bool
18	home_ownership_Mortgag	577 non-null	bool
19	home_ownership_Mortgage	577 non-null	bool
20	home_ownership_OTHER	577 non-null	bool
21	home_ownership_OWEN	577 non-null	bool
22	home_ownership_Own	577 non-null	bool
23	home_ownership_Rent	577 non-null	bool
24	home_ownership_Renting	577 non-null	bool
25	home_ownership_rent	577 non-null	bool
26	loan_purpose_AUTO	577 non-null	bool
27	loan_purpose_Auto Loan	577 non-null	bool
28	loan_purpose_Debt Consolidation	577 non-null	bool
29	loan_purpose_Home Improvement	577 non-null	bool
30	loan_purpose_Medical	577 non-null	bool
31	loan_purpose_Personal	577 non-null	bool
32	loan_purpose_Small Business	577 non-null	bool
33	loan_purpose_debt_consolidation	577 non-null	bool
34	loan_purpose_home_imp	577 non-null	bool
35	loan_purpose_medial	577 non-null	bool
36	loan_purpose_personal	577 non-null	bool
37	delinquency_status_60 DPD	577 non-null	bool
38	delinquency_status_90+ DPD	577 non-null	bool
39	delinquency_status_CHARGE_OFF	577 non-null	bool
40	delinquency_status_CURRENT	577 non-null	bool
41	delinquency_status_Current	577 non-null	bool
42	delinquency_status_Defaulted	577 non-null	bool
43	delinquency_status_Late (30)	577 non-null	bool
44	delinquency_status_Late (60)	577 non-null	bool

dtypes: bool(39), float64(6)

memory usage: 53.5 KB

Logistic Regression Model Training

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```